

Library Management System

Phase III: Physical Design & Implementation

Hanouf Jaber Alzahrani

ID: 4211511

System Description

System Scope & Core Functionality

The **Library Management System (LMS)** is a centralized relational database solution designed to digitize and streamline daily library operations, replacing manual workflows with an automated, highly efficient system.

Book & Inventory Management: Organizes books across various academic categories, tracks physical availability, and assigns unique identifiers (**Book_ID** & **ISBN**).

User Profile Management: Maintains detailed records for **Students**, library **Employees**, and **Managers** with appropriate operational privileges.

Circulation & Transactions: Automates book borrowing and returning workflows, enforces due dates, and tracks financial late fines.

Study Room Reservation: Manages time-bound room bookings for students, ensuring scheduling accuracy and preventing room overlap conflicts.

Objectives of Phase III

TRANSITIONING FROM ABSTRACT LOGICAL DESIGN TO PHYSICAL DATABASE

Phase III marks the transition from abstract logical design to a fully functional, production-ready physical database. This phase bridges theory and practice by implementing the Library Management System in MySQL complete with real data types, integrity constraints, business rules, and sample data that mirrors actual library operations.

1

Physical Database Creation

Transform the logical ER design into a physical MySQL database, creating all 10 entity tables with carefully chosen data types appropriate for each attribute.

2

Data Integrity Constraints

Implement standard SQL constraints PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and DEFAULT to enforce entity and referential integrity across all tables.

3

Business Rule Enforcement

Encode domain specific business logic directly into the schema using CHECK constraints, ensuring that invalid data can never be inserted at the database level.

Objectives of Phase III

DATABASE POPULATION & QUERY EVALUATION

4

Database Population

Populate all tables with realistic sample data to simulate actual library operations, including students, books, employees, study rooms, and transaction records.

5

Query Evaluation

Validate the database design through advanced SQL queries using JOINS, aggregate functions, GROUP BY, HAVING, and nested subqueries to extract meaningful insights.

Physical Schema Implementation

DDL - DATA DEFINITION LANGUAGE

Database & Table Creation

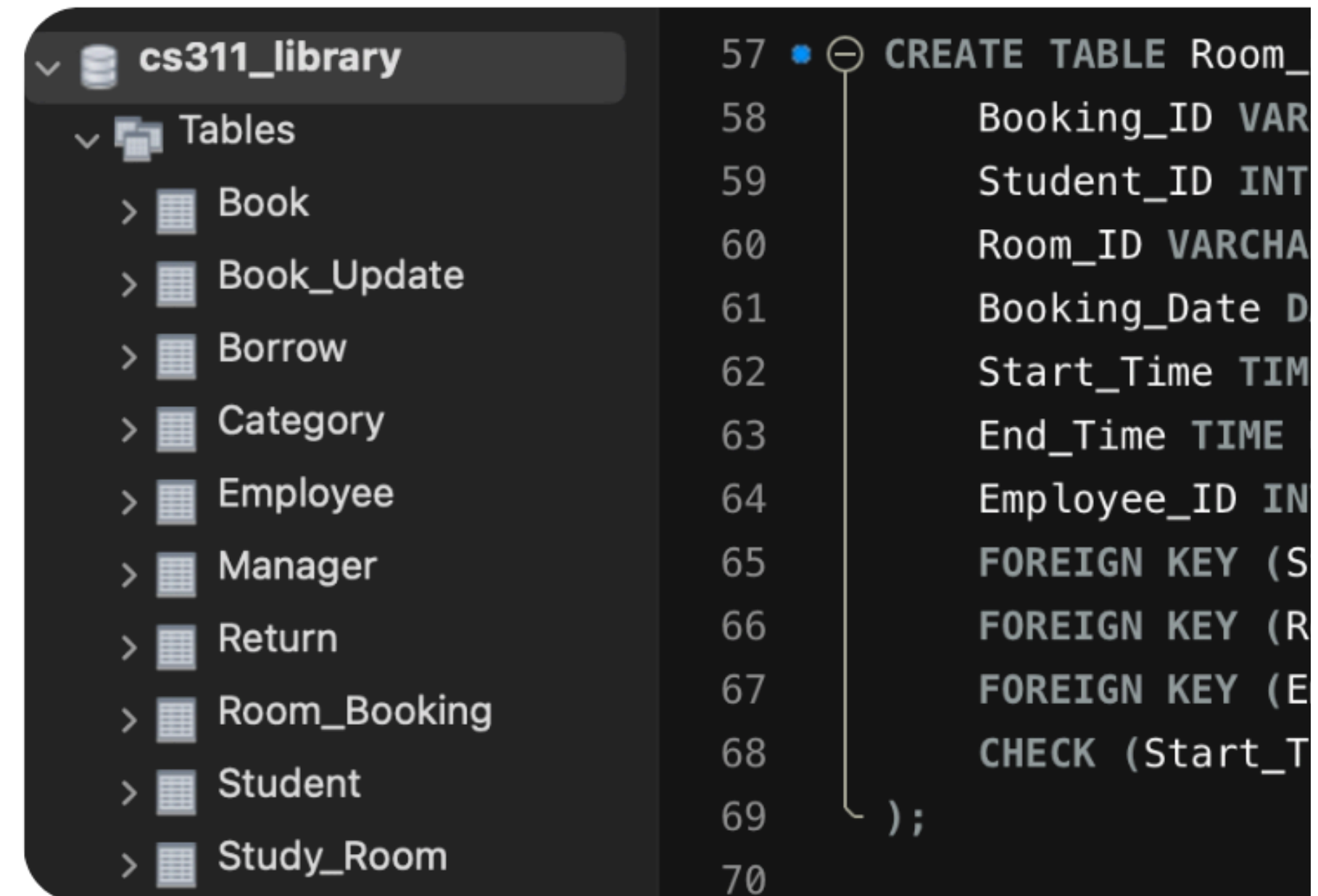
The cs311_library database was created as the container for all library-related entities. All 10 tables were defined using CREATE TABLE statements in MySQL Workbench, faithfully reflecting the logical ER model from Phase II.

Tables Created: Category, Book, Student, Employee, Study_Room, Borrow, Return, Book_Booking.

Data Type Selection

- **VARCHAR:** For string-based identifiers.
- **INT:** For numeric IDs and counts.
- **DATE / TIME:** For scheduling and deadlines.
- **DECIMAL:** For financial values (fines).
- **ENUM:** For fixed categorical values.

Careful data type selection ensures predictable behavior. E.g., using DECIMAL(10,2) for fines avoids floating-point precision errors that occur with FLOAT.



Implementing Standard Constraints

DATA INTEGRITY FOUNDATION

Data integrity is the cornerstone of a reliable database system. Five categories of standard SQL constraints were applied across all tables to guarantee that data remains accurate, consistent, and meaningful throughout its lifecycle.



PRIMARY KEY

Uniquely identifies every record within a table, preventing duplicate rows. Applied to all entity tables (e.g., Book_ID).



FOREIGN KEY

Maintains referential integrity between related tables by ensuring that a value in one table corresponds to an existing value.



UNIQUE

Prevents duplicate values in columns where uniqueness is a business requirement (e.g., Email in Student table, ISBN).



NOT NULL

Ensures that critical fields can never be left empty. Applied to essential attributes such as student names and booking dates.



DEFAULT

Assigns a sensible default value when no explicit value is provided (e.g., Room_Status defaults to 'Available').

Advanced Business Rule Implementation

CHECK CONSTRAINTS

Beyond standard constraints, CHECK constraints were used to encode domain-specific business rules directly into the database schema. This ensures that logically invalid data can never be inserted, updated, or persisted.

1. Time Logic

```
CHECK (Start_Time < End_Time)
```

Applied to Book_Booking table. A study room booking is only valid if the start time occurs before the end time. Rejecting impossible schedules.

2. Date Logic

```
CHECK (Due_Date >= Borrow_Date)
```

Applied to Borrow table. When a book is borrowed, the due date must be the same as or later than the borrow date, preventing past dates.

3. Financial Logic

```
CHECK (Fine >= 0)
```

Applied to Return table. Fines represent monetary penalties for late returns. Guarantees that recorded fines are non-negative.

CHECK constraints are evaluated on every INSERT and UPDATE operation, providing a robust, always-on layer of data validation.

Inserting Sample Data

DML - DATA MANIPULATION LANGUAGE

Population Strategy

After defining the schema, all tables were populated with realistic sample data designed to simulate genuine library operations (active borrowings, returned books with fines, and study rooms).

Data was inserted in a specific order to respect foreign key dependencies: parent tables first (Category, Student, Employee, Study_Room, Book), followed by transaction bridge tables (Borrow, Return, Rook_Booking).

Sample Data Highlights

- **Students:** Unique IDs, names, and emails.
- **Transactions:** Borrow and return records including fines (e.g., 15.50 SAR).
- **Books:** Diverse titles across categories with valid ISBNs.
- **Rooms:** Study rooms with bookings and time slots.

```
13      (2, 'Software Engineering'),
14      (3, 'Operating Systems'),
15      (4, 'Computer Networks'),
16      (5, 'Cybersecurity'),
17      (6, 'Artificial Intelligence'),
18      (7, 'Embedded Systems'),
19      (8, 'Data Science'),
20      (9, 'Web Development'),
21      (10, 'Algorithms & Data Structures');
22  -- 2. Manager Table (10 Rows)
23  DELETE FROM Manager;
24  INSERT INTO Manager (Manager_ID, Name, Email, Office) VALUES
25      (101, 'Dr. Kajal Nusratullah', 'kajal@yic.edu.sa', 'Main Office A-10'),
26      (102, 'Dr. Ahmed Al-Ahmadi', 'a.ahmadi@yic.edu.sa', 'Admin Office B-05'),
27      (103, 'Dr. Sarah Smith', 'ssmith@yic.edu.sa', 'Office A-11'),
28      (104, 'Dr. Ali Hassan', 'ahassan@yic.edu.sa', 'Office B-06'),
29      (105, 'Dr. Mona Zaki', 'mzaki@yic.edu.sa', 'Office A-12'),
30      (106, 'Dr. Yasser Omar', 'yomar@yic.edu.sa', 'Office B-07'),
31      (107, 'Dr. Huda Khaled', 'hkhaled@yic.edu.sa', 'Office A-13'),
32      (108, 'Dr. Sami Saeed', 'ssaeed@yic.edu.sa', 'Office B-08'),
33      (109, 'Dr. Nada Tariq', 'ntariq@yic.edu.sa', 'Office A-14'),
34      (110, 'Dr. Fahad Salem', 'fsalem@yic.edu.sa', 'Office B-09');
35  -- 3. Student Table (10 Rows)
36  DELETE FROM Student;
37  INSERT INTO Student (Student_ID, Name, Email) VALUES
38      (451500543, 'Layan Aljohani', 'layan@yic.edu.sa'),
39      (451500603, 'Alhanouf Aljohani', 'alhanouf@yic.edu.sa'),
40      (451500569, 'Wadeef Alalwani', 'wadeef@yic.edu.sa'),
41      (451500101, 'Sara Al-Ghamdi', 'sara.g@yic.edu.sa'),
42      (451500102, 'Noura Al-Otaibi', 'noura.o@yic.edu.sa'),
43      (451500103, 'Reem Al-Zahrani', 'reem.z@yic.edu.sa'),
44      (451500104, 'Mona Al-Harbi', 'mona.h@yic.edu.sa');
```


Derived Attributes

Dynamic calculation & Database Optimization

Derived attributes are intentionally **not stored physically** in relational tables to adhere to Third Normal Form (3NF) and prevent data redundancy and update anomalies. Instead, they are calculated dynamically using SQL functions and queries.

1

Total Fines (**Total_Fines**):

- Calculated dynamically based on overdue days:
`DATEDIFF(Return_Date, Due_Date) * Daily_Penalty.`
- Aggregated per student/employee using `SUM(Fine)`.

2

Booking Duration (**Duration**):

- Calculated from room reservation slots:
`TIMEDIFF(End_Time, Start_Time).`

3

Active Borrowed Books Count (**Active_Count**):

Evaluated dynamically via `COUNT(*)` on unreturned books in the Borrow table

Retrieving Borrowing Records

QUERY 1 OF 3

Objective

Demonstrates the use of SELECT with multiple JOIN operations to retrieve a comprehensive view of borrowing activity. Displays student names, book titles, and due dates.

SQL Features Used

- SELECT to specify output columns.
- JOIN (Student -> Borrow) linking via Student_ID.
- JOIN (Borrow -> Book) linking via Book_ID.

Why Multiple JOINS?

Without JOINS, we only see foreign key IDs. JOINS resolve those IDs into human-readable names and titles.

Expected Output

```
SELECT s.Name AS Student_Name, b.Title AS Book_Title, br.Borrow_Date, br.Due_Date
FROM Student s
JOIN Borrow br ON s.Student_ID = br.Student_ID
JOIN Book b ON br.Book_ID = b.Book_ID;
```

Student_Name	Book_Title	Borrow_Date	Due_Date	
Ahmed Al-Ghamdi	Python for Data Analysis	2026-07-15	2026-07-22	
Amal Al-Otaibi	Software Engineering Principles	2026-07-25	2026-08-01	
Ali Al-Zahrani	Computer Networking: Top-Down Approach	2026-07-26	2026-08-02	
Amal Al-Harbi	Artificial Intelligence: A Modern Approach	2026-07-27	2026-08-03	
Amal Al-Mutairi	Embedded Systems Architecture	2026-07-28	2026-08-04	
Amal Al-Shehri	Learning Web Design	2026-07-29	2026-08-05	
Amal Al-Kahlani	Introduction to Algorithms	2026-07-30	2026-08-06	
Amal Aljohani	Database System Concepts	2026-07-20	2026-07-27	
Amal Alalwani	Computer Security Principles	2026-07-10	2026-07-17	
Amal Aljohani	Operating System Concepts	2026-07-21	2026-07-28	

Financial Tracking by Employee

QUERY 2 OF 3

This query showcases SQL aggregation for financial reporting. It calculates the total fines processed by each employee, filtering to show only those who collected fines > zero.

- 1. **JOIN Tables:** Join Employee and Return on Employee_ID.
- 2. **GROUP BY:** Group results by employee name.
- 3. **SUM() & HAVING:** Calculate total fine and filter HAVING SUM(Fine) > 0.

Sample Result

```
SELECT e.Name AS Employee_Name, SUM(r.Fine) AS Total_Fines_
FROM Employee e
JOIN `Return` r ON e.Employee_ID = r.Employee_ID
GROUP BY e.Name
HAVING Total_Fines_Collected > 0;
```

1:8

  Filter Rows: Export: 

Name	Total_Fines_Collect...	
ubaie	15.50	
Dosari	25.00	
ahatani	5.00	
zi	10.00	
hammari	15.00	

Identifying Active Room Bookers

QUERY 3 OF 3

Objective

Identifies students who have actively booked a study room, returning their names and emails. Demonstrates a nested subquery to filter results based on related records.

SQL Features Used

- SELECT to retrieve Name and Email.
- WHERE ... IN to filter students.
- **Subquery:** A nested SELECT on Room_Booking returning Student_IDs.

Why Use a Subquery?

A subquery provides a clean way to express "find students who have made a booking" without requiring a JOIN. Useful for checking existence.

How It Works

The inner query executes first, returning a list of Student_IDs. The outer query then selects the name and email of any student whose ID appears in that list.

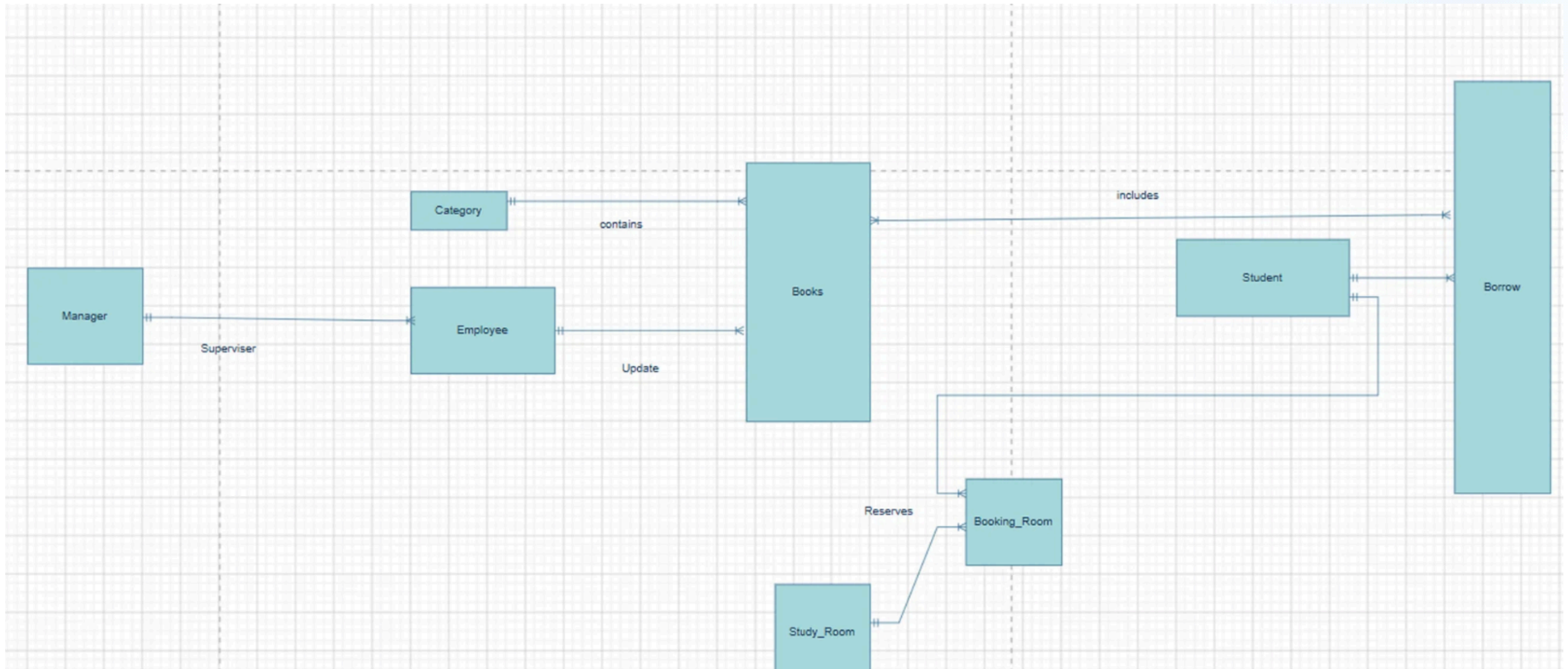
```
2 FROM Student
3 WHERE Student_ID IN (
4     SELECT Student_ID
5     FROM Room_Booking
6 );
```

Result Grid 1:9 Filter Rows: Search Export:

Name	Email
Layan Alihani	layan@vic.edu.sa

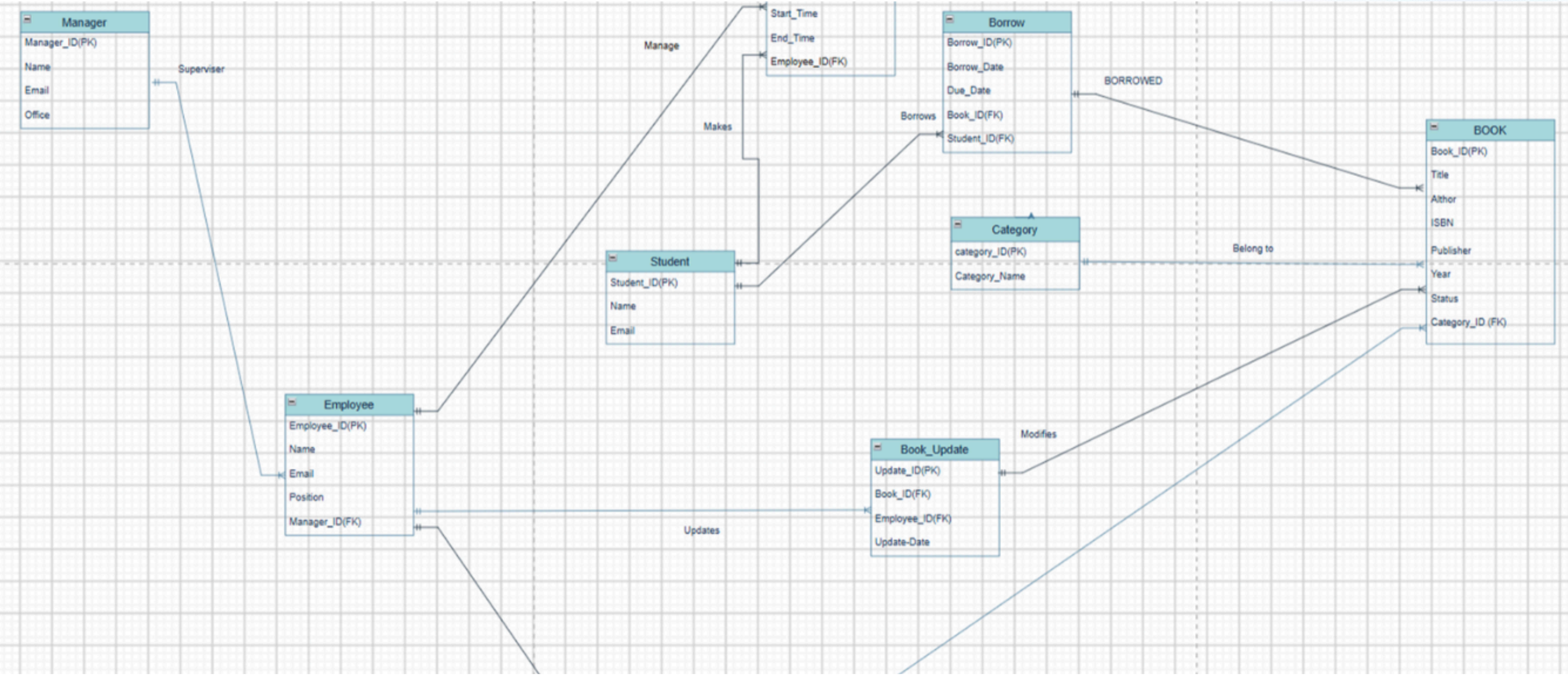
Entity Relationship Diagram

INITIAL ERD



Entity Relationship Diagram

FINAL ERD



Live System Demonstration

SWITCHING TO MYSQL WORKBENCH

We will now transition to a live demonstration in MySQL Workbench, executing the discussed queries in real time. This demonstration will showcase:



1. Schema Verification

Browsing the cs311_library database to show all 10 tables, data types, and constraints as defined.



2. Data Population

Running SELECT queries to confirm sample data insertion and constraint enforcement.



3. Query Execution

Running all three evaluation queries live (JOINS, aggregations, subqueries) displaying result sets.

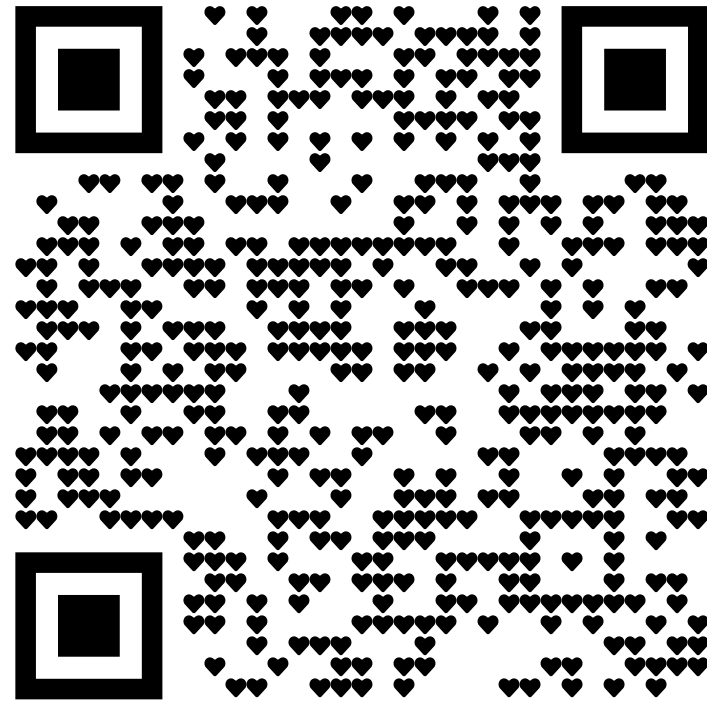


4. Constraint Testing

Attempting to insert invalid data (e.g., negative fines) to demonstrate CHECK constraints rejecting it.

Project Repository

Scan to Explore the Code



GitHub

Thank You

Questions & Feedback Welcome

10

Tables Created

All entities from the ER model
implemented.

5 & 3

Constraints & Checks

Business rules enforced at the
database level.

3

Queries Evaluated

Advanced SQL queries tested and
validated live.